

Mini E-commerce Distribuído

Relatório Técnico — Fundamentos de Computação Concorrente, Paralela e Distribuída

Disciplina:	Fund. de Computação Concorrente, Paralela e Distribuída
Linguagem:	Python 3.11 + Flask
Comunicação:	HTTP/REST via requests
Armazenamento:	Arquivos JSON por serviço
Autenticação:	JWT (HS256) + bcrypt

1. Comunicação entre Microsserviços

A comunicação foi implementada via **HTTP/REST** usando a biblioteca *requests* do Python. O API Gateway (porta 8000) atua como único ponto de entrada: recebe as requisições do cliente, valida headers e repassa para o microsserviço correto mantendo os headers originais (incluindo o Authorization com o JWT).

O fluxo típico de uma requisição autenticada é:

Cliente → **API Gateway :8000** → **Microsserviço :5001/5002/5003**
(valida rota + healthcheck) (valida JWT internamente)

A comunicação interna de replicação (primária → réplica de produtos) também usa HTTP POST com o header X-Internal: true para distinguir chamadas internas de chamadas de clientes externos.

2. Estratégia de Consistência na Replicação

Foi adotada **consistência forte (strong consistency)** para escritas no Serviço de Produtos.

Como funciona: ao receber um POST /products, a instância primária (porta 5002) persiste o produto localmente e, antes de responder ao cliente, faz uma chamada síncrona para a réplica (porta 5012) via POST /products com o mesmo payload. Caso a réplica não responda ou retorne erro, a escrita local é revertida (rollback) e o cliente recebe um erro 503. O sucesso só é confirmado quando **ambas as réplicas persistiram** o dado.

Leituras são distribuídas em round-robin pelo gateway entre a primária e a réplica, pois ambas estão sempre sincronizadas após qualquer escrita bem-sucedida.

Justificativa: para um e-commerce, inconsistências de estoque ou catálogo podem gerar pedidos inválidos. A consistência forte garante que todos os leitores vejam o mesmo estado imediatamente após uma escrita, ao custo de maior latência e da indisponibilidade em caso de falha da réplica.

3. Comportamento com Falha no Serviço de Pedidos

O sistema foi projetado para **degradação parcial**: cada microsserviço é independente. Se o Serviço de Pedidos (porta 5003) cair:

Serviço	Status com Pedidos fora	Impacto
Usuários :5001	✓ Continua funcionando	Login e registro normais
Produtos :5002	✓ Continua funcionando	Listagem e criação normais
Pedidos :5003	✗ Indisponível	503 em POST/GET /orders
Gateway :8000	✓ Detecta via heartbeat	Registra falha em log

O gateway detecta a falha em até 10 segundos (2 ciclos de heartbeat de 5s) e passa a retornar 503 para qualquer requisição direcionada ao serviço de pedidos, enquanto os demais continuam operando.

4. JWT e Controle de Acesso por Role

No login, o Serviço de Usuários gera um token JWT assinado com HS256 contendo:

```
{ "userId": "1", "email": "admin@loja.com", "role": "admin", "exp": <timestamp> }
```

O gateway repassa o header Authorization: Bearer <token> intacto para o microserviço de destino. O Serviço de Produtos decodifica o token usando a mesma chave secreta (JWT_SECRET) e verifica o campo role:

- Se role == 'admin': POST /products é autorizado.
- Se role == 'user': retorna **403 Forbidden** antes de processar qualquer dado.
- Token ausente ou expirado: retorna **401 Unauthorized**.

Um usuário comum — mesmo que intercepte a requisição — não pode forjar o campo role sem conhecer a chave secreta, pois a assinatura HMAC-SHA256 invalida qualquer alteração no payload.

5. Limitações em Relação a um Sistema Real

- **Armazenamento em JSON:** Arquivos JSON não oferecem transações ACID, controle de concorrência ou índices. Em produção, usa-se PostgreSQL, MongoDB ou similar com suporte a transações distribuídas.
- **Replicação síncrona acoplada:** A réplica de produtos causa acoplamento temporal: se ela estiver lenta, toda escrita fica lenta. Sistemas reais usam WAL, Raft ou Paxos para replicação tolerante a falhas parciais.
- **Gateway sem autoscaling:** O gateway é um único ponto de falha (SPOF). Em produção, usa-se Kubernetes + múltiplas réplicas do gateway com load balancer (Nginx, Traefik, AWS ALB).
- **Heartbeat simplificado:** O mecanismo verifica apenas se o serviço responde ao /health. Não detecta degradação de desempenho, erros silenciosos ou partição de rede parcial (split-brain).
- **JWT sem revogação:** Tokens JWT emitidos não podem ser revogados antes da expiração. Em produção, usa-se uma blacklist em Redis ou tokens de curta duração com refresh tokens.
- **Sem TLS entre serviços:** A comunicação interna ocorre em HTTP puro. Em produção, toda comunicação seria sobre HTTPS/mTLS para evitar ataques de interceptação na rede interna.

- **Sem circuit breaker:** Não há padrão Circuit Breaker (Hystrix, Resilience4j) para evitar cascata de falhas quando um serviço dependente está degradado.

*Implementação didática para a disciplina de Fundamentos de Computação Concorrente, Paralela e Distribuída.
Código disponível nos diretórios gateway/, users/, products/ e orders/.*